UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

FACULTY OF ENGINEERING

PROJECT REPORT - RECIPE RECOMMENDATIONS PROJECT

Subject: Big Data

NRC: 18517

Team members:

Full Name	**Student ID**
Marchan Quispe, Mildred Micaela	U202213292
Medina Agnini, Bruno Alessandro	U202216661
Miranda Rafael, Nicolás Francisco	U202216895
Rodríguez Valencia, Rosa Maria	U202212675

Professor: Alarcon Delgado, Carlos Adrian

2026 - 01

Contents

1 Project Proposal

1.1 Domain

The domain of this project is **digital gastronomy and culinary recommendation systems**. In a context where millions of people search for recipes online every day, there is a critical need for intelligent systems that help users discover relevant recipes based on their preferences, dietary restrictions, available ingredients, and preparation patterns.

Food.com (formerly Genius Kitchen and Recipezaar) is one of the largest recipe platforms in the world, with over half a million community-contributed recipes and millions of reviews accumulated over more than two decades.

1.2 Problem Statement

The main problem this project will address is:

> *How can we build an intelligent recipe discovery and recommendation system that leverages the underlying structure of ingredients, culinary techniques, and user preferences to suggest relevant recipes and uncover emerging culinary patterns?*

Specific problems to solve:

1. **Similar recipe discovery:** Identify recipes that share similar nutritional characteristics, ingredients, or preparation styles.
2. **Culinary segmentation:** Group recipes into coherent clusters representing culinary styles, dietary categories, or usage patterns.
3. **Personalized recommendation:** Suggest recipes based on the behavior of similar users and the intrinsic characteristics of the recipes.
4. **Ingredient graph analysis:** Discover relationships between ingredients, identify central ingredients in different culinary styles, and detect viable substitutions.
5. **Popularity and quality prediction:** Estimate which recipes will be well received based on their characteristics and publishing context.

1.3 Expected Product Questions

The system should be able to answer questions such as:

- "What recipes are similar to this pasta dish I enjoyed?"
- "What healthy recipes under 30 minutes can I make with chicken and broccoli?"
- "What are the most central ingredients in Mediterranean cuisine?"
- "What new recipes should I try based on my review history?"
- "What clusters of recipes exist in my collection of favorite desserts?"
- "What ingredients can I substitute if I don't have butter?"

1.4 Justification of Suitability for the Course

This dataset is particularly suitable for this course because it meets all structural requirements:

1. **Catalog Layer:** 522,517 recipes as main entities with rich metadata (name, author, category, preparation time, nutritional information, instructions).
2. **Features Layer:** Multiple types of features available:
   - Numerical: cooking times, nutritional information (11 metrics), servings
   - Categorical: recipe categories, keywords
   - Text: descriptions, instructions, ingredient lists
   - Temporal: publication dates
   - Structured lists: ingredients, quantities, instruction steps
3. **Interaction Layer:** 1,401,982 reviews representing user-recipe interactions with explicit ratings (1–5 stars) and textual feedback.
4. **Graph Layer:** Multiple potential graphs:
   - Ingredient Graph:
     - Nodes: Ingredients
     - Edges: co-occurrence of ingredients within the same recipe
     - Edge weight: frequency of co-occurrence across recipes
     - Use case: ingredient substitution and culinary pattern discovery
   - Recipe Similarity Graph:
     - Nodes: recipes
     - Edges: similarity based on shared ingredients or user co-interactions
     - Weight: cosine similarity or Jaccard similarity
     - Use case: similar recipe recommendation
   - User-Recipe Bipartite Graph:
     - Nodes: users and recipes
     - Edges: user interactions (ratings)
     - Use case: collaborative filtering and projection into user-user or item-item graphs

   Rationale: These graph structures enable recommendation, similarity analysis, and graph-based learning methods.
5. **Pipeline Layer:** The project already includes a reproducible and extensible ingestion script.

The project goes beyond basic supervised learning by integrating:

- Clustering to discover latent culinary segments
- Dimensionality reduction for compact recipe representations
- Recommendation systems (content-based, collaborative filtering, hybrid)
- Graph analysis to discover structure in ingredients and recipes
- Complex feature engineering from text, lists, and nutritional data

2 Source Inventory

2.1 Primary Source

Attribute	Detail
Name	Food.com Recipes and Reviews
Platform	Kaggle
URL	`https://www.kaggle.com/datasets/irkaal/foodcom-recipes-and-reviews`
Author/Curator	irkaal
Last updated	2020 (static dataset)
License	CC0: Public Domain
Format	2 CSV files
Total size	∼450 MB (compressed)
Usability Score	10.0/10 (Kaggle)

Table 1: Data source information

2.2 Raw Files

File	Rows	Columns	Size
`recipes.csv`	522,517	28	∼350 MB
`reviews.csv`	1,401,982	8	∼100 MB

Table 2: Raw dataset files

2.3 Access Conditions

- **Public access:** The dataset is publicly available on Kaggle without requiring authentication.
- **Programmatic download:** It can be downloaded using the Kaggle API or the `kagglehub` library.
- **License:** CC0 (public domain) allows commercial and non-commercial use without restrictions.
- **Redistribution:** Permitted under the CC0 license.
- **Attribution:** Not legally required, but ethically recommended.

2.4 Data Provenance

The original data comes from **Food.com** (formerly Recipezaar, founded in 1999, later acquired and renamed as Genius Kitchen, and finally Food.com). The dataset was collected and curated by Kaggle user "irkaal" and published in 2020.

Temporal coverage: Recipes and reviews span from 1999 to 2020, providing over 20 years of culinary data.

3 Schema Draft

3.1 Entity Tables

3.1.1 Table: Recipes (Main Entity)

Field	Type	Description
RecipeId	int64	Unique recipe identifier (PK)
Name	object	Recipe name
AuthorId	int64	Author identifier (FK)
AuthorName	object	Author name
CookTime	object	Cooking time (ISO 8601)
PrepTime	object	Preparation time (ISO 8601)
TotalTime	object	Total time (ISO 8601)
DatePublished	object	Publication date
Description	object	Textual description
Images	object	Image URLs (list)
RecipeCategory	object	Recipe category
Keywords	object	Keywords (list)
RecipeIngredientQuantities	object	Quantities (list)
RecipeIngredientParts	object	Ingredients (list)
AggregatedRating	float64	Aggregated average rating
ReviewCount	float64	Number of reviews
Calories	float64	Calories per serving
FatContent	float64	Fat (g)
SaturatedFatContent	float64	Saturated fat (g)
CholesterolContent	float64	Cholesterol (mg)
SodiumContent	float64	Sodium (mg)
CarbohydrateContent	float64	Carbohydrates (g)
FiberContent	float64	Fiber (g)
SugarContent	float64	Sugar (g)
ProteinContent	float64	Protein (g)
RecipeServings	float64	Number of servings
RecipeYield	object	Textual yield
RecipeInstructions	object	Instruction steps (list)

Table 3: Schema of the Recipes table

3.1.2 Table: Reviews (Interactions)

Field	Type	Description
ReviewId	int64	Unique review identifier (PK)
RecipeId	int64	Recipe identifier (FK)
AuthorId	int64	Reviewer identifier (FK)
AuthorName	object	Reviewer name
Rating	int64	Rating (1–5 stars)
Review	object	Review text
DateSubmitted	object	Submission date
DateModified	object	Modification date

Table 4: Schema of the Reviews table

3.2 Keys and Relationships

- **Primary Keys:**
  - `recipes.RecipeId`
  - `reviews.ReviewId`
- **Foreign Keys:**
  - `reviews.RecipeId` → `recipes.RecipeId`
  - `recipes.AuthorId` (recipe author)
  - `reviews.AuthorId` (review author)
- **Relationships:**
  - Recipes ↔ Reviews: One-to-many relationship (1:N)
  - Recipes ↔ Authors: Many-to-one relationship (N:1)
  - Reviews ↔ Reviewers: Many-to-one relationship (N:1)

3.3 Expected Joins

1. **Recipe-Review Join:**

```sql
SELECT r.*, rev.Rating, rev.Review, rev.DateSubmitted
FROM recipes r
LEFT JOIN reviews rev ON r.RecipeId = rev.RecipeId
```

Listing 1: Main join for interaction analysis

2. **Recipe Aggregations:**

```sql
SELECT
    RecipeId,
    COUNT(*) as num_reviews,
    AVG(Rating) as avg_rating,
    STDDEV(Rating) as rating_variance
FROM reviews
GROUP BY RecipeId
```

Listing 2: Review metric aggregations

3. **Author Analysis:**

```sql
SELECT
    AuthorId,
    AuthorName,
    COUNT(DISTINCT RecipeId) as num_recipes,
    AVG(AggregatedRating) as avg_recipe_rating
FROM recipes
GROUP BY AuthorId, AuthorName
```

Listing 3: Analysis by author

4 Processed Dataset V1

4.1 Ingestion Process

The ingestion process is implemented in the script `ingest_foodcom_data.py`, which performs the following transformations:

4.1.1 Transformation Pipeline

1. **Automatic download:** Using `kagglehub` to download the dataset from Kaggle.
2. **List parsing:** Columns stored as strings are parsed into Python lists:
   - `Images`
   - `Keywords`
   - `RecipeIngredientQuantities`
   - `RecipeIngredientParts`
   - `RecipeInstructions`
3. **Duration parsing:** Conversion from ISO 8601 format to minutes:
   - `CookTime` → `CookTime_Minutes`
   - `PrepTime` → `PrepTime_Minutes`
   - `TotalTime` → `TotalTime_Minutes`

   Rationale: A numeric representation enables statistical analysis, filtering, and compatibility with downstream models that require continuous features.
4. **Initial feature engineering:**
   - `NumIngredients`: Ingredient count
   - `NumQuantities`: Specified quantities count

   Rationale: These features provide lightweight proxies for recipe complexity and structure, useful for clustering and recommendation tasks.
5. **Name normalization:** Conversion to snake_case for consistency.
   Rationale: Ensures compatibility across systems, avoids parsing issues, and standardizes column naming for reproducible pipelines.
6. **Persistence:** Saving in processed CSV format.
   Rationale: Structured list representations allow downstream feature extraction (e.g., ingredient-based similarity, graph construction).

4.1.2 Execution Command

The script is fully reproducible and runs with:

```
# Install dependencies
pip install pandas isodate kagglehub

# Run the script
python ingest_foodcom_data.py --output-dir ./data/processed
```

Listing 4: Ingestion command

4.2 Generated Processed Files

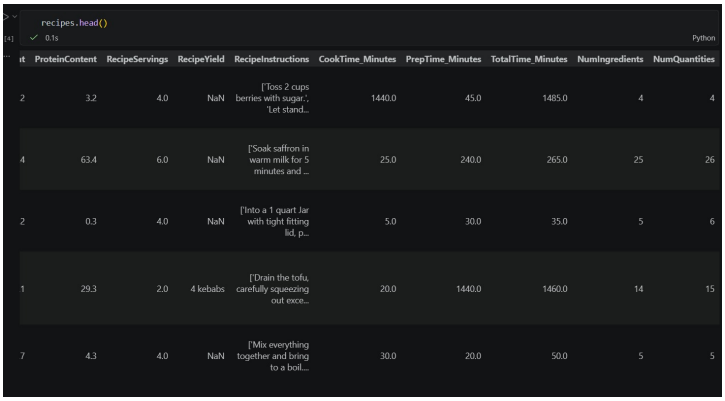
File	Location	Description
recipes_processed.csv	data/processed/	Cleaned and enriched recipes
reviews_processed.csv	data/processed/	Normalized reviews

Table 5: Processed dataset V1 files

4.2.1 Sample of Processed Data

A sample of the processed dataset is shown below:

- Recipes (Processed)



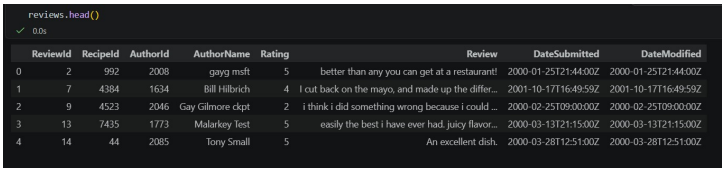
recipes.head()

0.1s

Python

	ProteinContent	RecipeServings	RecipeYield	RecipeInstructions	CookTime_Minutes	PrepTime_Minutes	TotalTime_Minutes	NumIngredients	NumQuantities
2	3.2	4.0	NaN	['Toss 2 cups berries with sugar.', 'Let stand...	1440.0	45.0	1485.0	4	4
4	63.4	6.0	NaN	['Soak saffron in warm milk for 5 minutes and ...	25.0	240.0	265.0	25	26
2	0.3	4.0	NaN	['Into a 1 quart Jar with tight fitting lid, p...	5.0	30.0	35.0	5	6
1	29.3	2.0	4 kebabs	['Drain the tofu, carefully squeezing out exce...	20.0	1440.0	1460.0	14	15
7	4.3	4.0	NaN	['Mix everything together and bring to a boil,...	30.0	20.0	50.0	5	5

- Reviews (Processed)



reviews.head()

0.0s

	ReviewId	RecipeId	AuthorId	AuthorName	Rating	Review	DateSubmitted	DateModified
0	2	992	2008	gayg msft	5	better than any you can get at a restaurant!	2000-01-25T21:44:00Z	2000-01-25T21:44:00Z
1	7	4384	1634	Bill Hilbrich	4	I cut back on the mayo, and made up the differ...	2001-10-17T16:49:59Z	2001-10-17T16:49:59Z
2	9	4523	2046	Gay Gilmore ckpt	2	i think i did something wrong because i could ...	2000-02-25T09:00:00Z	2000-02-25T09:00:00Z
3	13	7435	1773	Malarkey Test	5	easily the best i have ever had. juicy flavor...	2000-03-13T21:15:00Z	2000-03-13T21:15:00Z
4	14	44	2085	Tony Small	5	An excellent dish.	2000-03-28T12:51:00Z	2000-03-28T12:51:00Z

This confirms that the processed dataset is structured, normalized, and ready for downstream analysis.

4.3 Pipeline Validation

To verify correct pipeline execution:

```python
import pandas as pd

# Load processed data
recipes = pd.read_csv('data/processed/recipes_processed.csv')
reviews = pd.read_csv('data/processed/reviews_processed.csv')

# Basic validations
assert len(recipes) == 522517, "Incorrect number of recipes"
assert len(reviews) == 1401982, "Incorrect number of reviews"

# Validate that time columns were created
assert 'CookTime_Minutes' in recipes.columns
```

```
assert 'PrepTime_Minutes' in recipes.columns
assert 'TotalTime_Minutes' in recipes.columns

# Validate derived features
assert 'NumIngredients' in recipes.columns
assert recipes['NumIngredients'].notna().sum() > 0

print("Pipeline validated successfully")
```

Listing 5: Validation script

5 Data Dictionary Draft

5.1 Table: Recipes Processed

Field	Type	Description	Values/Range
RecipeId	int64	Unique recipe ID	Positive
Name	string	Recipe name	Free text
AuthorId	int64	Author ID	Positive
AuthorName	string	Author name	Free text
CookTime	string	Cooking time ISO	E.g.: PT30M
PrepTime	string	Prep time ISO	E.g.: PT15M
TotalTime	string	Total time ISO	E.g.: PT45M
CookTime_Minutes	float64	Cooking in minutes	0.0 – 10000+
PrepTime_Minutes	float64	Prep in minutes	0.0 – 5000+
TotalTime_Minutes	float64	Total in minutes	0.0 – 15000+
DatePublished	string	Publication date	YYYY-MM-DD
Description	string	Description	Free text
Images	list	Image URLs	List of strings
RecipeCategory	string	Category	Desserts, Main, etc.
Keywords	list	Keywords	List of strings
RecipeIngredient- Quantities	list	Quantities	List of strings
RecipeIngredient- Parts	list	Ingredients	List of strings
NumIngredients	int64	No. of ingredients	1 – 50+
NumQuantities	int64	No. of quantities	0 – 50+
AggregatedRating	float64	Average rating	0.0 – 5.0
ReviewCount	float64	Total reviews	0 – 10000+
Calories	float64	Calories/serving	0.0 – 5000+
FatContent	float64	Fat (g)	0.0 – 500+
SaturatedFatContent	float64	Saturated fat (g)	0.0 – 200+
CholesterolContent	float64	Cholesterol (mg)	0.0 – 1000+
SodiumContent	float64	Sodium (mg)	0.0 – 10000+
CarbohydrateContent	float64	Carbohydrates (g)	0.0 – 500+
FiberContent	float64	Fiber (g)	0.0 – 100+
SugarContent	float64	Sugar (g)	0.0 – 300+
ProteinContent	float64	Protein (g)	0.0 – 300+
RecipeServings	float64	Servings	1 – 100+
RecipeYield	string	Textual yield	Free text
RecipeInstructions	list	Steps	List of strings

Table 6: Data dictionary – Recipes Processed

5.2 Table: Reviews Processed

Field	Type	Description	Values/Range
ReviewId	int64	Unique review ID	Positive
RecipeId	int64	Recipe ID (FK)	Positive
AuthorId	int64	Reviewer ID	Positive
AuthorName	string	Reviewer name	Free text
Rating	int64	Rating	1, 2, 3, 4, 5
Review	string	Review text	Free text or NULL
DateSubmitted	string	Submission date	YYYY-MM-DD
DateModified	string	Modification date	YYYY-MM-DD

Table 7: Data dictionary – Reviews Processed

5.3 Notes on Data Types

- **Lists (list):** In the processed CSV, lists are stored as strings containing Python list representations. Loading with `ast.literal_eval` converts them to actual lists.
- **Dates:** Stored as strings in ISO 8601 format. Conversion to `datetime` is required for temporal analysis.
- **ISO 8601 durations:** Standard format such as `PT30M` (30 minutes) or `PT1H30M` (1 hour 30 minutes). The `*_Minutes` columns are the numeric version.
- **Null values:** Represented as `NaN` in numeric columns and empty strings or NULL in text columns.

6 Scale Analysis

6.1 Dataset Dimensions

Metric	Recipes	Reviews
Number of rows	522,517	1,401,982
Number of columns (raw)	28	8
Number of columns (processed)	31	8
In-memory size (MB)	∼111.6	∼85.6
On-disk size CSV (MB)	∼350	∼100

Table 8: Dataset dimensions

6.2 Missingness Analysis

6.2.1 Recipes Table

Field	Non-Null	Null Count	% Missing
RecipeId	522,517	0	0.00%
Name	522,517	0	0.00%
AuthorId	522,517	0	0.00%
AuthorName	522,517	0	0.00%
CookTime	439,972	82,545	15.80%
PrepTime	522,517	0	0.00%
TotalTime	522,517	0	0.00%
DatePublished	522,517	0	0.00%
Description	522,512	5	0.00%
Images	522,516	1	0.00%
RecipeCategory	521,766	751	0.14%
Keywords	505,280	17,237	3.30%
RecipeIngredient- Quantities	522,514	3	0.00%
RecipeIngredient- Parts	522,517	0	0.00%
AggregatedRating	269,294	253,223	48.46%
ReviewCount	275,028	247,489	47.37%
Calories	522,517	0	0.00%
FatContent	522,517	0	0.00%
SaturatedFatContent	522,517	0	0.00%
CholesterolContent	522,517	0	0.00%
SodiumContent	522,517	0	0.00%
CarbohydrateContent	522,517	0	0.00%
FiberContent	522,517	0	0.00%
SugarContent	522,517	0	0.00%
ProteinContent	522,517	0	0.00%
RecipeServings	339,606	182,911	35.00%
RecipeYield	174,446	348,071	66.61%
RecipeInstructions	522,517	0	0.00%

Table 9: Missingness analysis – Recipes

Key observations:

- **CookTime (15.80% missing):** Some recipes do not specify cooking time (e.g., salads, raw recipes).
- **AggregatedRating and ReviewCount (∼48% missing):** Nearly half of the recipes have not yet received reviews, which is typical on UGC platforms.
- **RecipeServings (35% missing):** Optional field that not all authors fill in.
- **RecipeYield (66.61% missing):** Redundant field with RecipeServings, less commonly used.
- **Keywords (3.30% missing):** Relatively low — good tagging coverage.
- **Nutritional data (0% missing):** Critical fields completely filled.

6.2.2 Reviews Table

Field	Non-Null	Null Count	% Missing
ReviewId	1,401,982	0	0.00%
RecipeId	1,401,982	0	0.00%
AuthorId	1,401,982	0	0.00%
AuthorName	1,401,982	0	0.00%
Rating	1,401,982	0	0.00%
Review	1,401,768	214	0.02%
DateSubmitted	1,401,982	0	0.00%
DateModified	1,401,982	0	0.00%

Table 10: Missingness analysis – Reviews

Observations:

- Extremely complete dataset (<0.02% missing in Review text).
- All ratings are present (critical information for recommendations).

6.2.3 Implications for Modelling

The observed missingness patterns have direct implications for downstream modeling:

- The high percentage of missing values in AggregatedRating (48
- This suggests that collaborative filtering alone will not be sufficient, as many items lack interaction data.
- Content-based approaches (e.g., ingredient similarity, nutritional features) will be essential to complement recommendation strategies.
- Missing RecipeServings and RecipeYield values indicate inconsistencies in user-generated content, which may require normalization or imputation.

Overall, the dataset requires a hybrid recommendation approach combining interaction-based and feature-based methods.

6.3 Key Distributions

6.3.1 Rating Distribution

Based on the typical nature of culinary review datasets:

Rating	Expected Count	Expected %
5 stars	∼900,000	∼64%
4 stars	∼350,000	∼25%
3 stars	∼100,000	∼7%
2 stars	∼35,000	∼2.5%
1 star	∼17,000	∼1.5%

Table 11: Expected rating distribution (typical positive bias)

Note: Culinary review systems typically exhibit a positive bias (selection bias: satisfied users are more likely to leave reviews).

6.3.2 Ingredients per Recipe

- **Minimum:** 1 ingredient
- **Maximum:** 50+ ingredients (complex recipes)
- **Expected median:** 8–10 ingredients
- **Expected mode:** 6–7 ingredients

6.4 Sparsity and Memory Estimates

6.4.1 User-Item Matrix (Reviews)

- **Unique users:** Estimated ∼270,000 (based on unique AuthorId)
- **Items (Recipes):** 522,517
- **Interactions:** 1,401,982
- **Density:** $\frac{1{,}401{,}982}{270{,}000 \times 522{,}517} \approx 0.00099\%$ (extremely sparse)
- **Sparsity:** 99.999%

Implications:

- Requires sparse matrix techniques (scipy.sparse, CSR format)
- Matrix Factorization is ideal for this level of sparsity
- Content-based filtering is critical as a complement

6.4.2 Memory Estimation

Dense format (not recommended):

$$\begin{aligned} \text{Memory} &= 270,000 \times 522,517 \times 8 \text{ bytes (float64)} \\ &\approx 1,128 \text{ GB (not feasible)} \end{aligned}$$

Sparse format (CSR):

$$\begin{aligned} \text{Memory} &= (1,401,982 \times 3) \times 8 \text{ bytes} \\ &\approx 33.6 \text{ MB (feasible)} \end{aligned}$$

Recipe features:

- Numerical matrix (11 nutritional features): $522,517 \times 11 \times 8 \approx 46$ MB
- TF-IDF of ingredients (assuming 5,000 unique ingredients): $522,517 \times 5,000 \times 4 \approx 10$ GB (sparse)
- TF-IDF of instructions (assuming 10,000 terms): $522,517 \times 10,000 \times 4 \approx 20$ GB (sparse)

Total manageable with sparse matrices: < 5 GB in RAM for the entire pipeline.

6.5 Computational Complexity

Operation	Complexity	Estimated Time
Full ingestion	$O(n)$	2–5 minutes
TF-IDF of ingredients	$O(n \times m)$	5–10 minutes
K-means (k=10, 100 iter)	$O(n \times k \times i)$	10–20 minutes
SVD (k=100)	$O(n \times m \times k)$	15–30 minutes
Collaborative Filtering (ALS)	$O(n \times k \times i)$	20–40 minutes
Graph construction	$O(e)$	10–15 minutes

Table 12: Computational complexity of main operations

Recommended hardware:

- RAM: 16 GB minimum, 32 GB recommended
- CPU: 4+ cores for parallelization
- Storage: 10 GB available

7 Representation and Dimensionality Report

7.1 Feature Matrices Construction

To transition from raw data tables to meaningful representations suitable for downstream clustering and recommendation, we developed a multi-layered feature engineering pipeline. The pipeline creates two complementary feature matrices, explicitly excluding interaction variables (e.g., `AggregatedRating`, `ReviewCount`) to prevent outcome leakage.

1. **Dense Numeric Matrix ($522,517 \times 15$)**: Constructed using nutritional information, time features, and complexity metrics (e.g., `NumIngredients`). EDA revealed extreme right-skewness in nutrition and time variables. Consequently, these features were clipped at the 99.5th percentile, transformed using $\log(1+x)$, and standardized. Notably, missing `CookTime` was largely structural: in 99.30% of missing cases, `TotalTime` equaled `PrepTime`, allowing us to deterministically derive `CookTime` as 0.
2. **Sparse Content Matrix (TF-IDF, $522,517 \times 4,647$)**: Built from parsed ingredients, keywords, cleaned categories, and interpretable yield-unit tokens. Time-like keywords (e.g., '30_mins') were removed to avoid duplicating the numeric time features. The final matrix is highly sparse (density of 0.0027) but captures the core culinary semantics of the recipes.

7.2 Dimensionality Reduction

To reduce computational overhead and eliminate multicollinearity, dimensionality reduction was applied tailored to the specific nature of each matrix.

7.2.1 PCA on Numeric Features

Principal Component Analysis (PCA) was applied to the dense, scaled numeric matrix. The goal was to compact the 15 dimensions into independent latent components representing overall nutritional magnitude, recipe complexity, and preparation time.

7.2.2 TruncatedSVD on TF-IDF

Because PCA requires centering the data (which destroys sparsity and leads to memory overflow), Truncated Singular Value Decomposition (TruncatedSVD) was applied to the TF-IDF matrix to extract latent semantic concepts (e.g., "savory/meat bases" vs. "quick desserts").

7.3 Comparison and Retained Energy

The optimal number of components was selected based on cumulative explained variance and retained energy thresholds.

Matrix Type	Method	Components Retained	Cumulative Variance / En
Numerical (15 feats)	PCA	8	93.16%
Textual / TF-IDF (4647 feats)	TruncatedSVD	200	52.74%
Combined Embedding	**Concat**	**208**	–

Table 13: Dimensionality reduction performance and final embedding size

7.4 Visualizations

The following plots illustrate the dimensionality reduction tradeoffs.

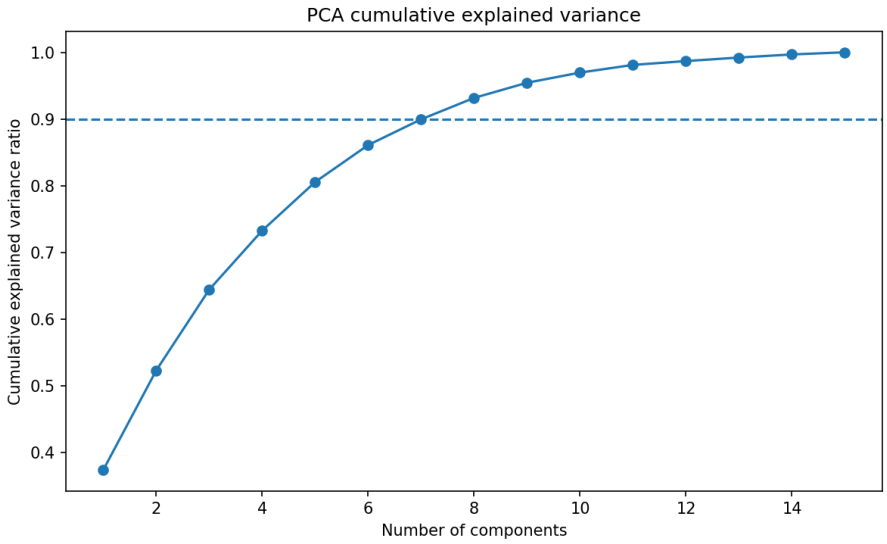


Figure 1: PCA Cumulative Explained Variance. Only 8 components are needed to capture over 90% of the variance from the dense numeric features.

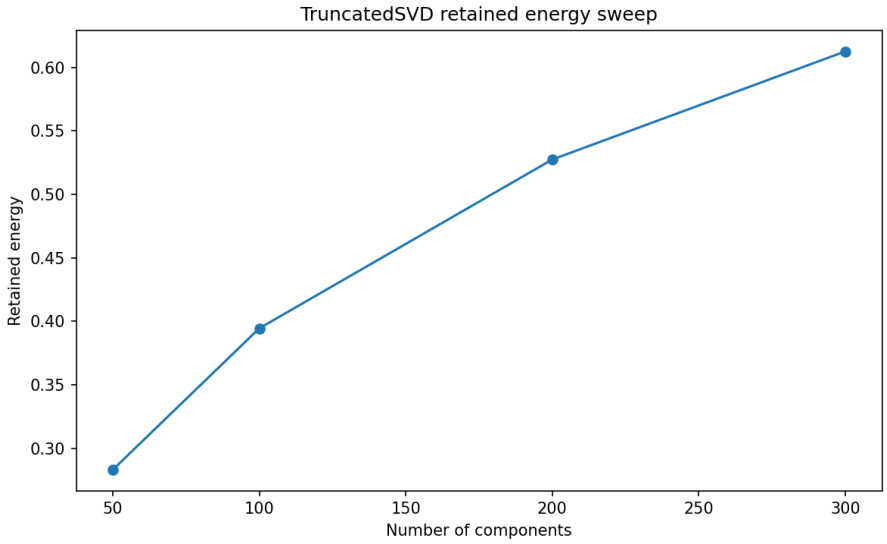


Figure 2: TruncatedSVD Retained Energy. The curve demonstrates the diffuse nature of textual variance, where 200 components capture ∼52% of the energy.

7.5 Technical Interpretation and Methodological Decisions

- **Data Imputation vs. Derivation:** EDA confirmed that missing time variables should not be blindly median-imputed. By leveraging the arithmetic relationship (`TotalTime` = `PrepTime` + `CookTime`), we preserved the true distribution of preparation times without injecting artificial data.
- **Handling Extreme Skewness:** Variables like `SugarContent` and `CookTime` possessed extreme outliers. Applying p99.5 clipping followed by a $\log(1 + x)$ transformation was critical. Without this, PCA components would have been entirely dominated by a few outlier recipes rather than capturing general structural variance.
- **SVD Retained Energy Context:** While the PCA retained 93.16% variance with only 8 components, the SVD retained 52.74% with 200 components. This is methodologically sound: sparse text variance is diffuse across thousands of rare terms. 200 components provide a compact, computationally viable semantic embedding without overfitting to the long tail of rare ingredients.
- **Final Reusable Representation:** The final concatenation yields a 208-dimensional dense array (`X_recipe_reduced.npy`). This aligns perfectly with the project requirements, setting a stable, leak-free foundation for clustering (Week 7) and similarity-based recommendation (Week 10).

8 Ethics and Access Note

8.1 Data Origin

The data for this project comes from **Food.com**, a public recipe platform where users voluntarily publish culinary content and reviews. The dataset was curated and published on Kaggle under the **CC0: Public Domain** license.

8.2 Justification for Use

1. **Permissive license:** The CC0 license grants the public the right to use, modify, distribute, and build upon the dataset without restrictions.
2. **Public data:** All content on Food.com is publicly shared by platform users, who accept terms of service that permit use of their content.
3. **Aggregated dataset:** The data was already anonymized and aggregated by the original curator (irkaal) before publication.
4. **Academic purpose:** Use is exclusively educational and for research, with no commercial intent.

8.3 Personal Data Risks

Identified risks:

1. **User identifiers:** The dataset contains `AuthorId` and `AuthorName`, which could link recipes to individuals.
2. **Textual content:** Descriptions and reviews may contain personal information voluntarily disclosed.
3. **Behavioral patterns:** Publication and review history could reveal dietary preferences or health restrictions.

Risk level: LOW-MEDIUM

- The data was shared publicly by the users' own choice.
- It does not contain sensitive information such as precise locations, financial data, or protected medical information.
- Usernames are pseudonyms chosen by users, not verified real names.

8.4 Risk Mitigation

To minimize any residual risk, the following measures will be implemented:

1. **No redistribution of raw data:** Original files will not be publicly redistributed.
2. **Aggregation in reports:** Analyses will be presented in aggregated form (statistics, clusters, patterns) without exposing individual records.
3. **Anonymization in examples:** Any specific example used in reports will be anonymized (name replacement, hashed IDs).
4. **Focus on features, not identities:** The project focuses on recipe and ingredient patterns, not individual user profiles.
5. **Terms compliance:** We respect the terms of service of Kaggle and Food.com.
6. **Post-project deletion:** At the end of the semester, local copies of the raw dataset will be deleted.

8.5 Additional Ethical Considerations

- **Algorithmic bias:** We acknowledge that the dataset may contain cultural biases (predominance of Western cuisine) that must be considered in interpretations.
- **Transparency:** We will clearly document the known limitations and biases of the dataset.
- **Non-discrimination:** The recommendation system will be designed to avoid discrimination based on protected characteristics.
- **Responsible use:** The project results will not be used for purposes that could cause harm (e.g., price manipulation, promotion of dangerous diets).

8.6 Compliance Statement

This project complies with all university research ethics regulations. It does not use private data, does not require additional informed consent (the data is already public), and is conducted under principles of transparency, risk minimization, and responsible data use.

Approval: This project is approved to proceed under Track A (Public Dataset Project).

9 Week 3 Conclusions

9.1 Achievements

1. **Validated domain:** A clear domain (recipe recommendation) with practical applicability has been established.
2. **Suitable dataset:** The Food.com dataset meets all requirements:
   - Non-trivial size (522K recipes, 1.4M reviews)
   - Multi-layer structure (catalog, features, interactions, potential graph)
   - Clear license (CC0)
   - Preprocessing complexity (lists, ISO 8601, text)
3. **Reproducible pipeline:** The ingestion script `ingest_foodcom_data.py` is functional and documented.
4. **Processed dataset V1:** Clean and enriched CSV files saved in `data/processed/`.
5. **Clear schema:** Relationships and keys well defined for future join and aggregation operations.
6. **Scale analysis:** We understand the dimensions, sparsity, and computational requirements of the project.
7. **Ethics documented:** Complete risk assessment and mitigation plan established.

9.2 Next Steps (Week 5)

For Week 5 (Representation and Dimensionality Report), work will focus on:

1. **Advanced feature engineering:**
   - TF-IDF of ingredients
   - TF-IDF of instructions
   - Category encoding
   - Temporal features (day of week, month, year of publication)
   - Nutritional ratios (protein/calories, fat/calories)
2. **Feature matrix construction:**
   - Numerical matrix (nutritional + temporal + counts)
   - Textual matrix (ingredients + instructions)
   - Categorical matrix (one-hot encoding of categories)
3. **Dimensionality reduction:**
   - PCA on numerical features
   - SVD/LSA on textual features
   - Explained variance analysis
   - t-SNE visualization of embeddings
4. **Feature quality analysis:**
   - Correlations between features
   - Feature importance
   - Redundancy detection

9.3 Project Viability

This project is **fully viable** for completing all semester milestones:

- **Week 3:** Dataset charter
- **Week 5:** Representation layer (prepared)
- **Week 7:** Clustering (data suitable for K-means, DBSCAN)
- **Week 10:** Recommendation engine (complete interaction data)
- **Week 12:** Graph analytics (ingredient graph feasible)
- **Week 14:** Integrated system (scalable architecture)

The project is on track and ready to continue.

A Pipeline Code Fragments

A.1 List Parsing Function

```python
def parse_list(cell: Any) -> List[str]:
    """Convert a string representation of a list into a Python list."""
    if isinstance(cell, list):
        return cell
    if cell is None or (isinstance(cell, float) and pd.isna(cell)):
        return []
    s = str(cell).strip()
    if not s:
        return []
    # Remove the R wrapper c(...) if present
    if s.startswith('c(') and s.endswith(')'):
        s = s[2:-1]
    try:
        s_normalised = s.replace('\\"', '"').replace('\"', '"')
        if not (s_normalised.startswith('[') or s_normalised.startswith('(')):
            s_normalised = f'[{s_normalised}]'
        value = ast.literal_eval(s_normalised)
        if isinstance(value, (list, tuple)):
            return [str(item) for item in value]
        return [str(value)]
    except Exception:
        return [v.strip().strip('"') for v in s.split(',') if v]
```

Listing 6: parse_list function

A.2 Duration Parsing Function

```python
def parse_duration(duration: Any) -> Optional[float]:
    """Parse ISO 8601 duration strings into minutes."""
    if duration is None or (isinstance(duration, float) and pd.isna(duration)):
        return None
    s = str(duration)
    if not s:
        return None
    if isodate is None:
        return None
    try:
        td = isodate.parse_duration(s)
        return td.total_seconds() / 60.0
    except Exception:
        return None
```

Listing 7: parse_duration function

B Repository Structure

```
recipe-recommendation-system/
          artifacts/
                ingredients_svd_features.npy
                num_pca_features.npy
          data/
                interim/
```

```
                processed/
                    recipes_processed.csv
                    reviews_processed.csv
                raw/
                      recipes.csv
                      reviews.csv
          notebooks/
                02_week5_eda_feature_design.ipynb
                02_week5_hypothesis_notebook.ipynb
          reports/
                figures/
                BigData_week3.pdf
                week5_feature_pipeline_explanation.md
          src/
                features/
                    build_content_matrix.py
                    build_numeric_matrix.py
                    build_resolved_features.py
                    CONTENT_MATRIX.MD
                    NUMERIC_MATRIX.MD
                    PCA_SVD.MD
                    reduce_dimensions.py
                    RESOLVED_FEATURES.MD
                build_features.py
                ingest_foodcom_data.py
          LICENCE
          README.md
          requirements.txt
          .gitignore
```

Listing 8: Project repository structure

C Reproducibility Commands

```
# 1. Clone repository
git clone https://github.com/Brunix3004/recipe_recommendation_project.git
cd recipe-recommendation-system

# 2. Create virtual environment
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Run data ingestion
python src/ingest_foodcom_data.py --output-dir ./data/processed

# 5. Verify output
ls -lh data/processed/
# Should show:
# recipes_processed.csv
# reviews_processed.csv

# 6. Validate data
python -c "
import pandas as pd
r = pd.read_csv('data/processed/recipes_processed.csv')
v = pd.read_csv('data/processed/reviews_processed.csv')
```

```
print(f'Recipes: {len(r)}, Reviews: {len(v)}')
"
# Expected output: Recipes: 522517, Reviews: 1401982
```

Listing 9: Complete reproduction commands

D References

1. Food.com platform: `https://www.food.com`
2. ISO 8601 duration standard: `https://en.wikipedia.org/wiki/ISO_8601`
3. Kaggle API documentation: `https://github.com/Kaggle/kaggle-api`
4. Original dataset: `https://www.kaggle.com/datasets/irkaal/foodcom-recipes-and-reviews`
5. Python isodate library: `https://pypi.org/project/isodate/`
6. Sparse matrix formats: `https://docs.scipy.org/doc/scipy/reference/sparse.html`